

CS101-Quiz3-Review

Suting Chen

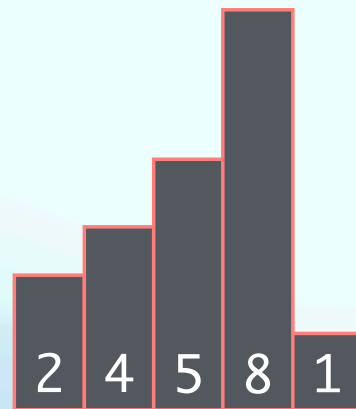
CS101-Quiz3-Review

Key Points

1. Insertion Sort — Swap vs. Move
2. Bubble Sort
3. Merge Sort

Insertion Sort

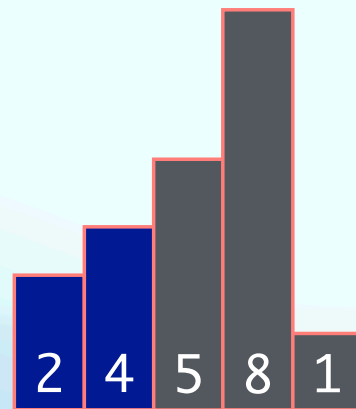
Implement with swap



Compare	Swap
0	0

Insertion Sort

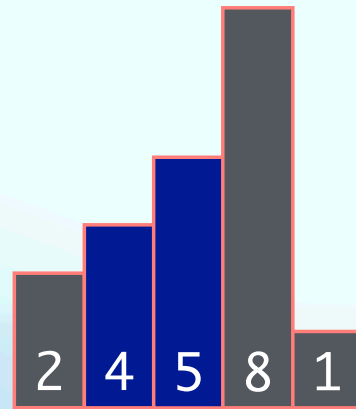
Implement with swap



Compare	Swap
1	0

Insertion Sort

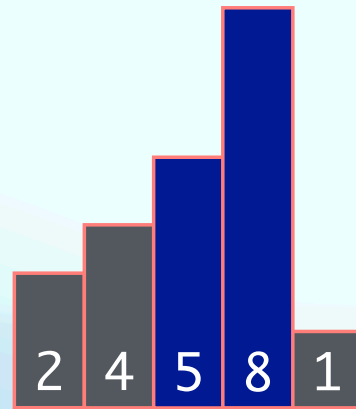
Implement with swap



Compare	Swap
2	0

Insertion Sort

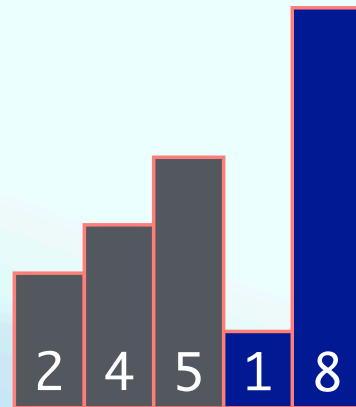
Implement with swap



Compare	Swap
3	0

Insertion Sort

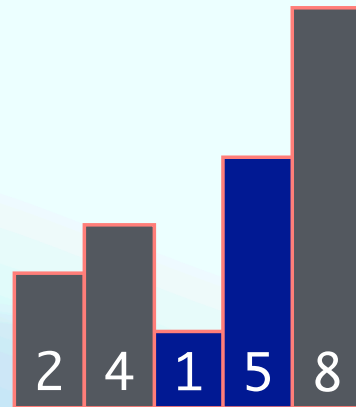
Implement with swap



Compare	Swap
4	1

Insertion Sort

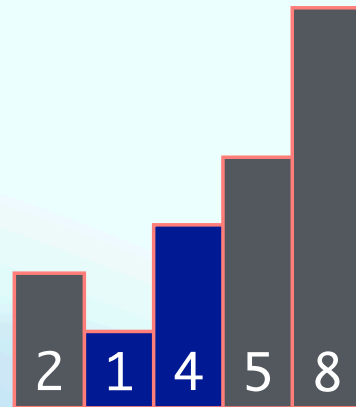
Implement with swap



Compare	Swap
5	2

Insertion Sort

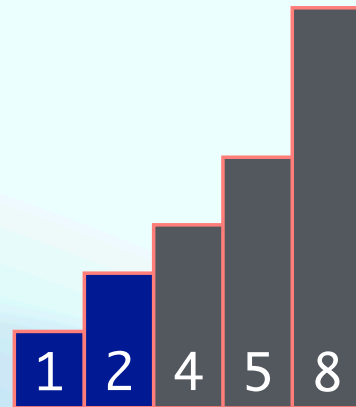
Implement with swap



Compare	Swap
6	3

Insertion Sort

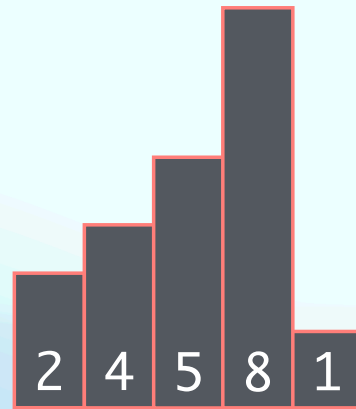
Implement with swap



Compare	Swap
7	4

Insertion Sort

Implement with **move**

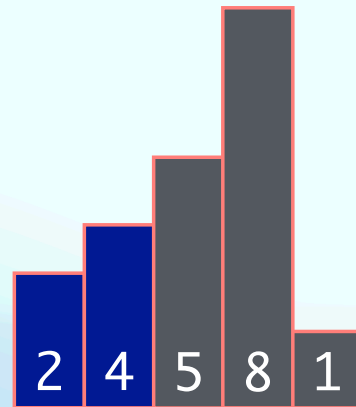


Compare	Move
0	0

4 Swaps

Insertion Sort

Implement with **move**

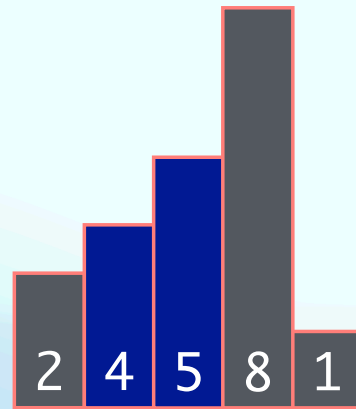


Compare	Move
1	0

4 Swaps

Insertion Sort

Implement with **move**

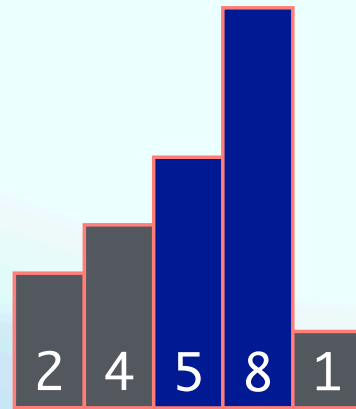


Compare	Move
2	0

4 Swaps

Insertion Sort

Implement with **move**

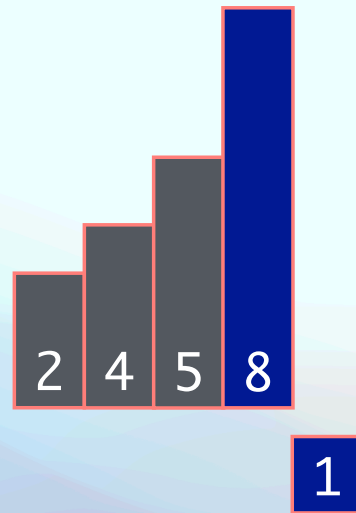


Compare	Move
3	0

4 Swaps

Insertion Sort

Implement with **move**

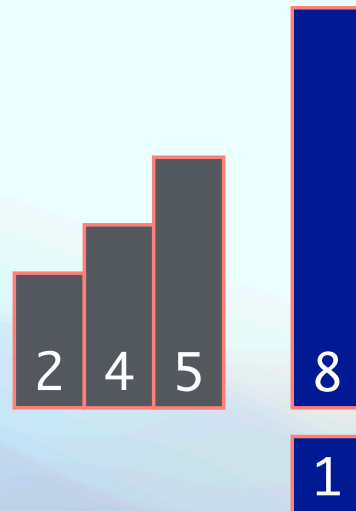


Compare	Move
4	1

4 Swaps

Insertion Sort

Implement with **move**

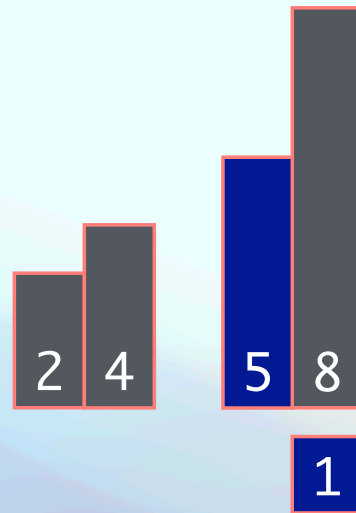


Compare	Move
4	2

4 Swaps

Insertion Sort

Implement with **move**

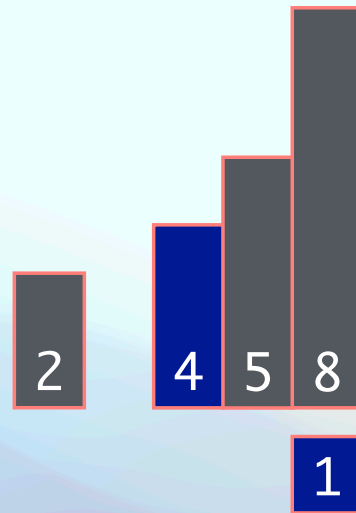


Compare	Move
5	3

4 Swaps

Insertion Sort

Implement with **move**

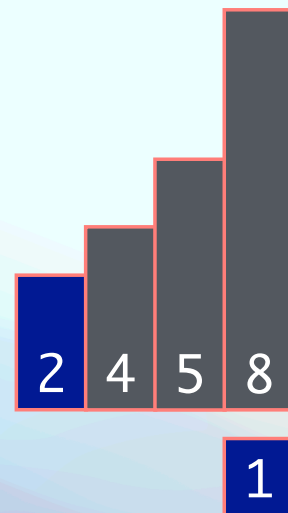


Compare	Move
6	4

4 Swaps

Insertion Sort

Implement with **move**

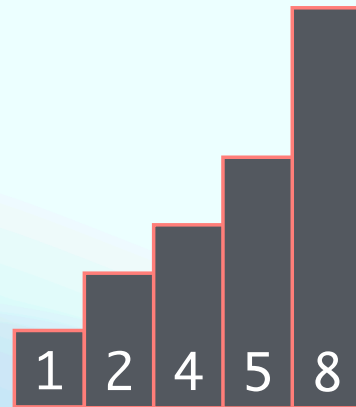


Compare	Move
7	5

4 Swaps

Insertion Sort

Implement with **move**



Compare	Move
7	6

4 Swaps

Insertion Sort

Swap vs move

```
1 void insertionSortBySwap(Vector& V)
2 {
3     for (int i = 1; i < V.size(); ++i) {
4         int j = i;
5         while (j > 0 and V[j] < V[j - 1]) {
6             swap(V[j], V[j - 1]);
7             j -= 1;
8         }
9     }
10 }
```

```
1 void insertionSortByMove(Vector &V)
2 {
3     for (int i = 1; i < V.size(); i++) {
4         int key = V[i];
5         int j = i - 1;
6         while (j ≥ 0 && V[j] > key) {
7             V[j + 1] = V[j];
8             j = j - 1;
9         }
10        V[j + 1] = key;
11    }
12 }
```

这里是说明实际上swap和move的数量是（几乎）一样的

Insertion Sort

Swap vs move

In C++, it is very likely that 1 swap takes 3 moves.

算法导论上只有move一种写法。实际上没有人会写基于swap的插入排序。

CS101-Quiz3-Review

Key Points

1. Insertion Sort — Swap vs. Move

2. Bubble Sort

3. Merge Sort

Bubble Sort

Comparing with insertion ...

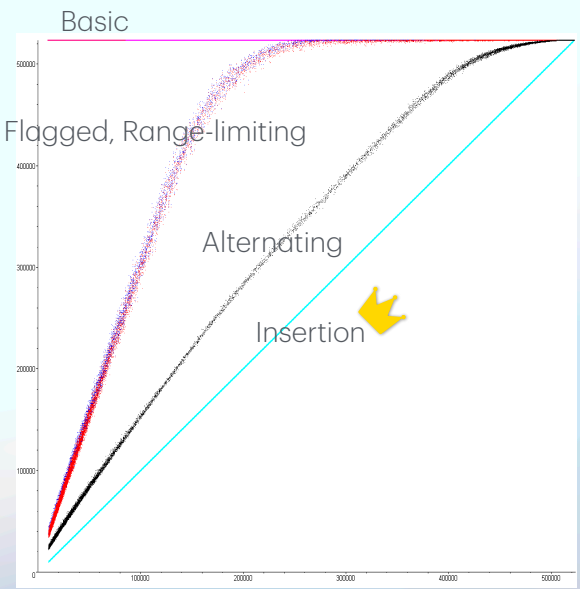
Significantly worse than insertion sort

其实是有一点优点的，这东西并行起来很快。
相比之下选择排序和插入排序都对并行不是很友好。

Bubble Sort

Improvements

Basic Bubble Sort	-
Flagged	If no swaps occur, stop
Range-limiting	Limit loops based on last swap location
Alternating	Bidirectional version of bubble sort



这里其实Range-limiting和Alternating都不是很重要。
特别是Range-limiting：听起来很fancy但是实际上提升很少（slides说了）

Sorting Algorithms

Conclusion

Case	Bubble	Insertion
Best	$O(n)$	$O(n)$
Worst	$O(n^2)$	$O(n^2)$
Average	$O(n^2)$	$O(n^2)$

1. If the number of inversions is d , the run time of insertion sort is $\Theta(n + d)$.
2. It is very difficult to estimate comparisons made by **optimized** bubble sort.
3. Both **in-place** and **stable**.

但是，这里可以保证bubble sort无论怎么优化都不比insertion的少。因为他本质上还是只能相邻的一个一个比较。
这里引用一句话，就是bubble sort几乎没有用，但是他有很多理论上可以研究的、好玩的东西。（slides上有类似的）

CS101-Quiz3-Review

Key Points

1. Insertion Sort — Swap vs. Move
2. Bubble Sort
- 3. Merge Sort**

Merge Sort

Key Points

1. Based on a strategy named **divide-and-conquer**.
2. Time complexity $\Theta(n \log n)$ in best, average, and worst cases.
3. Requires additional space: $\Theta(n)$.
4. (With basic array-like implementation) Not in-place.
5. Stable.

PA1的链表排序就是in-place的，所以其实一个算法的特性很大程度上取决于具体实现。

给出in-place的定义：an in-place algorithm is an algorithm that operates directly on the input data structure without requiring extra space proportional to the input size.

TL;DR: $O(N)$, 比如log

所以我们说merge sort不是inplace算法，是因为它涉及到拷贝数组进行merge，而不是因为他递归了。

Merge Sort

Time complexity analysis

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & n > 1 \end{cases}$$

Deal with two halves + Merging

Merge Sort

Time complexity analysis

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & n > 1 \end{cases}$$

Ways to solve a recurrence (递归式)

1. Substitution Method (代入法)
2. Recursion Tree (递归树法)
3. Master Theorem (主方法)

这些翻译很奇怪，但是他们是算法导论的东西。

递归树法课件上有，主定理后面会讲。因此这里写一个代入法。

Merge Sort

Time complexity analysis — **Proof by Substitution Method**

Recursion: $T(n) = 2 \times T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$

Assume: $T(n) = O(n \log n)$

Goal: For some $c > 0$, $T(n) \leq cn \log n$

$$T(n) \leq 2 \left(c \left\lfloor \frac{n}{2} \right\rfloor \log \left\lfloor \frac{n}{2} \right\rfloor \right) + n$$

$$\leq cn \log \left(\frac{n}{2} \right) + n$$

$$= cn \log n - cn \log 2 + n$$

$$= cn \log n - cn + n$$

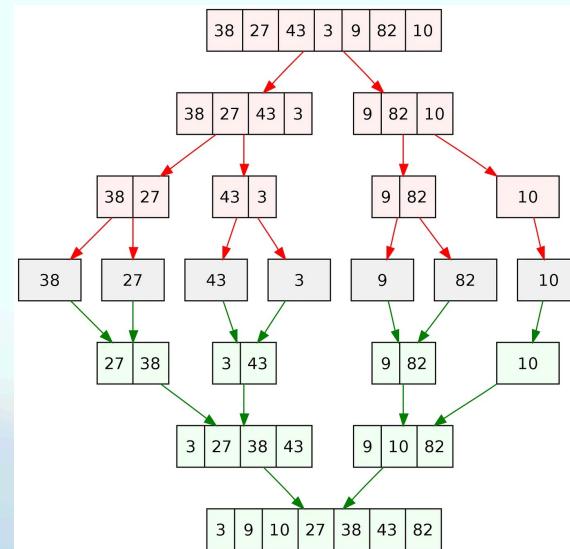
$$\leq cn \log n$$

Obviously, the statement is true as long as $c \geq 1$

这里不能算是数学归纳，数归自己也是能证明这个等式，但是算法导论没给。

Divide-and-conquer

Count inversions



Divide-and-conquer

Count inversions

3	27	38	43
---	----	----	----

9	10	82
---	----	----

Divide-and-conquer

Count inversions

3 27 38 43

9 10 82

Inversions
0

Divide-and-conquer

Count inversions

27 38 43

9 10 82

3

Inversions
0

Divide-and-conquer

Count inversions

27 38 43

9 10 82

3

Inversions

0

Divide-and-conquer

Count inversions

27 38 43

10 82

3 9

Inversions
0+3

Divide-and-conquer

Count inversions

27 38 43

10 82

3 9

Inversions
3

Divide-and-conquer

Count inversions

27 38 43

82

3 9 10

Inversions
3+3

Divide-and-conquer

Count inversions

27 38 43

82

3 9 10

Inversions
6

Divide-and-conquer

Count inversions

38 43

82

3 9 10 27

Inversions
6

Divide-and-conquer

Count inversions

43

82

3 9 10 27 38

Inversions

6

Divide-and-conquer

Count inversions

3	9	10	27	38	43	82
---	---	----	----	----	----	----

Inversions
6

1. 分解
2. 求解子问题
3. 合并

虽然课上我们使用了Enhance Merge Sort引出了这些分治相关的问题，但实际上分治是一个重要的思想，可以用于解决矩阵乘法、FFT等问题（这两个slides上面提到了），这跟mergesort并没有必然联系。

PA1 released

Check Piazza and OJ **NOW!**

截止到本slides完成，还有一半同学没有接受作业（在OJ上）。这里要催催。